

Progress Report

TrioTagaPayo — AMS_DBTC Project

Date	May 5, 2026
Reporter	James Neil M. Maano
Project	TrioTagaPayo AMS DBTC
Report Period	March 7, 2026 – May 5, 2026
Sprint / Period	Current Sprint

STAND-UP REPORT

What was achieved this period?

Previously completed (March 7 – April 25, 2026):

- Set up Backend folder/project structure and pushed to GitHub
- Implemented QR Code Attendance System (QRSession, QRScan, IQRService, QRSessionRepository)
- Integrated EmailJS Email Notification System with failure isolation
- Enhanced Attendance Repository with filter and bulk interfaces
- Added Global Exception Middleware with structured JSON error responses
- Configured Program.cs with all repositories, services, JWT, Swagger, CORS, admin seeding
- Implemented API Rate Limiting (login, global, qrscan policies)
- Implemented Refresh Token with Rotation (RefreshTokenHelper, POST /api/Auth/refresh)
- Implemented SignalR Real-time Notifications (AttendanceHub, NotifyHubAsync)
- Implemented JWT Cookie Session (HttpOnly cookies, logout, GET /api/Auth/me, ClaimsHelper)

NEW — May 5, 2026:

- Implemented Admin and Teacher Role-Based Account Hierarchy
 - Added TeacherId FK to User model — proper DB-level link between User and Teacher records
 - Configured User to Teacher FK relationship in AppDbContext with cascade SetNull on delete
 - Embedded teacher_id claim inside JWT so teachers can self-identify without extra DB calls
 - Added GetTeacherId() extension method to ClaimsHelper
 - Added ChangePasswordAsync to IAuthService and AuthService — any role can change own password
 - Block login and token refresh for disabled teachers (IsActive = false check in AuthService)
 - Fixed TeacherService to use TeacherId FK lookup — replaced fragile username string matching
 - Fixed CreateWithAccountAsync to use dto.Username and correctly set User.TeacherId
 - Added [Authorize(Roles = "Admin")] on all Teacher mutation endpoints (POST, PUT, PATCH, DELETE)

- Added GET /api/Teacher/me — Teacher reads own profile using TeacherId claim from JWT
 - Added POST /api/Auth/change-password endpoint for all roles
 - Added ChangePasswordDto with validation
 - Applied EF Core Migration: AddTeacherIdToUser (manually applied via Neon SQL Editor)
 - Resolved Production Deployment Issues on Render
 - Fixed connection string format mismatch — Render was using PostgreSQL URL format; corrected to Npgsql format
 - Identified and added missing Render environment variables: EmailJS__PublicKey, EmailJS__ServiceId, EmailJS__TemplateId
 - Rotated exposed EmailJS API keys and Neon DB password after accidental credential exposure
 - Cleaned up hardcoded key references from EmailJSHelper inline comments
 - Verified EmailJS template variables match backend payload (to_email, student_name, student_no, course_name, section, status, date, time_recorded, title)
 - Confirmed QR endpoints already implemented — communicated correct endpoint contracts to frontend team
-

What is the focus today?

- Verify login returns 200 with JWT token on live Render deployment
 - Run dotnet ef database update once network allows port 5432 access
 - Confirm Teacher role login works end-to-end after TeacherId FK migration
 - Test POST /api/Auth/change-password for both Admin and Teacher roles
 - Test GET /api/Teacher/me for logged-in Teacher role
 - Coordinate with frontend team on updated QR endpoint contracts
 - Push all remaining commits to GitHub branch Maano
-

What blockers exist?

- School/office network blocks port 5432 — cannot run dotnet ef database update locally
 - Workaround applied: TeacherId column added manually via Neon SQL Editor and migration registered in __EFMigrationsHistory
 - Render deployment returning 500 — root cause is password authentication failure; fix in progress via env var update
-

RETROSPECTIVE

What went well? (Wins/Successes)

- Role-based access control cleanly implemented with minimal changes — Admin and Teacher roles fully separated
- TeacherId FK approach is more robust than previous username string matching — eliminates a class of bugs
- Disabled teacher login block works at both login and token refresh level — no bypass possible
- Manual migration via Neon SQL Editor was a good workaround for network-blocked port 5432
- EmailJS template variables matched backend payload — no template rework needed
- Credential rotation was done quickly after exposure — public key, private key, and DB password all rotated

- Git commit history follows conventional commit format (feat, fix, chore) consistently
- Render environment variable structure identified and corrected (double underscore separator)

What didn't go well? (Problems/Challenges)

- Live credentials were accidentally exposed in chat — Neon password and EmailJS keys were visible
- Render was configured with PostgreSQL URL format instead of Npgsql format — caused 400 errors on login
- Missing EmailJS environment variables (PublicKey, ServiceId, TemplateId) were not set in Render
- School network blocks port 5432 — direct dotnet ef database update is not possible on this network
- EF migration had to be applied manually via SQL and __EFMigrationsHistory insert — risk of drift

What should we start doing? (Improvements)

- Never paste credentials, connection strings, or API keys into any chat or public channel
- Use a .env file locally and add it to .gitignore — never store secrets in appsettings.json committed to Git
- Test on mobile hotspot when school network blocks database ports
- Verify all Render environment variables are complete before testing live endpoints
- Write integration tests for role-based access (Admin vs Teacher endpoint restrictions)

What should we stop doing? (Waste/Inefficiencies)

- Stop using PostgreSQL URL format for Npgsql connection strings — always use the key=value Npgsql format
- Stop committing without checking git status first — verify which files are actually modified
- Avoid re-running migrations that already exist — check __EFMigrationsHistory before applying

What should we continue doing? (Best Practices)

- Maintain conventional commit format: feat, fix, chore, refactor, docs
- Keep role authorization at the controller level with [Authorize(Roles)] attributes
- Keep helper classes (ClaimsHelper, RefreshTokenHelper, RateLimitingHelper) in the Helpers folder
- Continue isolating email and notification failures from core business logic
- Maintain organized folder structure and feature-branch Git workflow (branch: Maano)
- Continue consistent progress updates and stand-up check-ins

Submitted by:

James Neil M. Maano

TrioTagaPayo Team

March 7, 2026 – May 5, 2026